

Lab 1 – Cryptography

General Instructions

- **Languages:** You may use **C, Java, or Python**. **Python is recommended** because it is the simplest for this lab.
- **One file per problem:** Submit *one code file* per problem (e.g., `p1_permutation.py`, `p2_hash.py`, `p3_rsa.py`).
- **Report & README:** Include a short **report (PDF)** with the outputs and explanations requested in each problem, and a **README** describing how to build/run each file.
- **Libraries policy:** You may use libraries for **gcd** and **modular inverse**, and for **SHA-256 in Problem 2 only**. Implement all other cryptographic and comparison logic yourself.

Policies on the Usage of LLM and AI Assistance

Students can use Large Language Models (LLMs) and AI-based tools, such as GitHub Copilot, to assist in completing their programming projects. The choice of LLM or AI tool is left entirely to the student. However, these tools may not always produce correct or optimal code. The student must ensure their final code is functional, accurate, and meets all project requirements. **As part of the project evaluation, students must submit any prompts they used to generate or refine their code. This submission should be included as a separate document accompanying the project submissions.**

Problem 1: Permutation Cipher (20 points)

Implement the permutation cipher encryption/decryption.

Task: Given the plaintext as

Q9fL3XvT8pR2mN7kD1sA6cH0yZ5uJ4eBqWn

and the key

7 1 4 5 2 3 6,

please write a program to output the ciphertext, and convert the ciphertext back to the plaintext.

Problem 2: Hash Functions (20 points)

Slides: Lecture 6, pages 16 and 19.

Use SHA-256 (e.g., Python's `hashlib.sha256`) to check file integrity.

Task 1: Computing and Storing the Hash

Create `message.txt` containing `Computer Security` (UTF-8, no trailing newline). Read the file, compute its SHA-256 hash, and save the hexadecimal digest in `reference_hash.txt`.

Task 2: Verifying File Integrity

Compare the file's current hash with the stored digest. Print `PASS` for a match or `FAIL` otherwise. Test the unchanged file, then replace its contents with `computer Security` and test again. Keep the stored digest unchanged. Print both hashes and both results.

In 2-3 sentences, explain how the comparison detects the modification and why storing a hash can save space compared with keeping a duplicate of a large file.

Problem 3: RSA Cipher Implementation (60 points)

You may use third-party libraries to calculate `gcd` or modular inverse. Python is the easiest language for this task.

Task 1: Deriving the Private Key

Let p , q , and e be three prime numbers. Let $n = p \times q$. We will use (e, n) as the public key. Please calculate the private key d .

The hexadecimal values of p , q , and e are listed below. It should be noted that although p and q used in this task are quite large numbers, they are not large enough to be secure. We intentionally make them small for the sake of simplicity. In practice, these numbers should be at least 512 bits long (the ones used here are only 128 bits).

```
p = F7E75FDC469067FFDC4E847C51F452DF
q = E85CED54AF57E53E092113E62F436F4F
e = 0D88C3
```

Task 2: Encryption

Let (e, n) be the public key. Please encrypt the message and generate the ciphertext:

"B7A19C3D5E2"

Task 3: Decryption

Please decrypt the ciphertext from Task 2 using the derived private key d .